

Data Engineers Already Solved Agentic AI's Reliability Problem

CONSTANCE MARTINEAU

AIRFLOW SUMMIT 2026



CONSTANCE MARTINEAU

STAFF PRODUCT MANAGER · ASTRONOMER

The data in this talk comes from **Astro**, the managed Airflow platform I work on. I used to write DAGs for a living. I sometimes cosplay as a **data scientist**.

ASTRONOMER

github.com/cmarteepants

linkedin.com/in/constance-martineau

A pipeline fails.
Someone asks **what**
changed.

POSSIBLE CAUSES

Schema drift

Late arrival

Env change

Something you shipped

You already know it (probably) **wasn't you.**

a column changed type

the file came late

an upstream team never read your DAG

Two things to take out of this room

01

WHAT BREAKS PIPELINES?

02

WHO SHOULD IMPLEMENT AI?

Somebody in your company is putting AI into production right now, and it is probably **not you. I think that's backwards.**

400 MILLION PRODUCTION PIPELINE RUNS

Astronomer managed platform

2025 calendar year

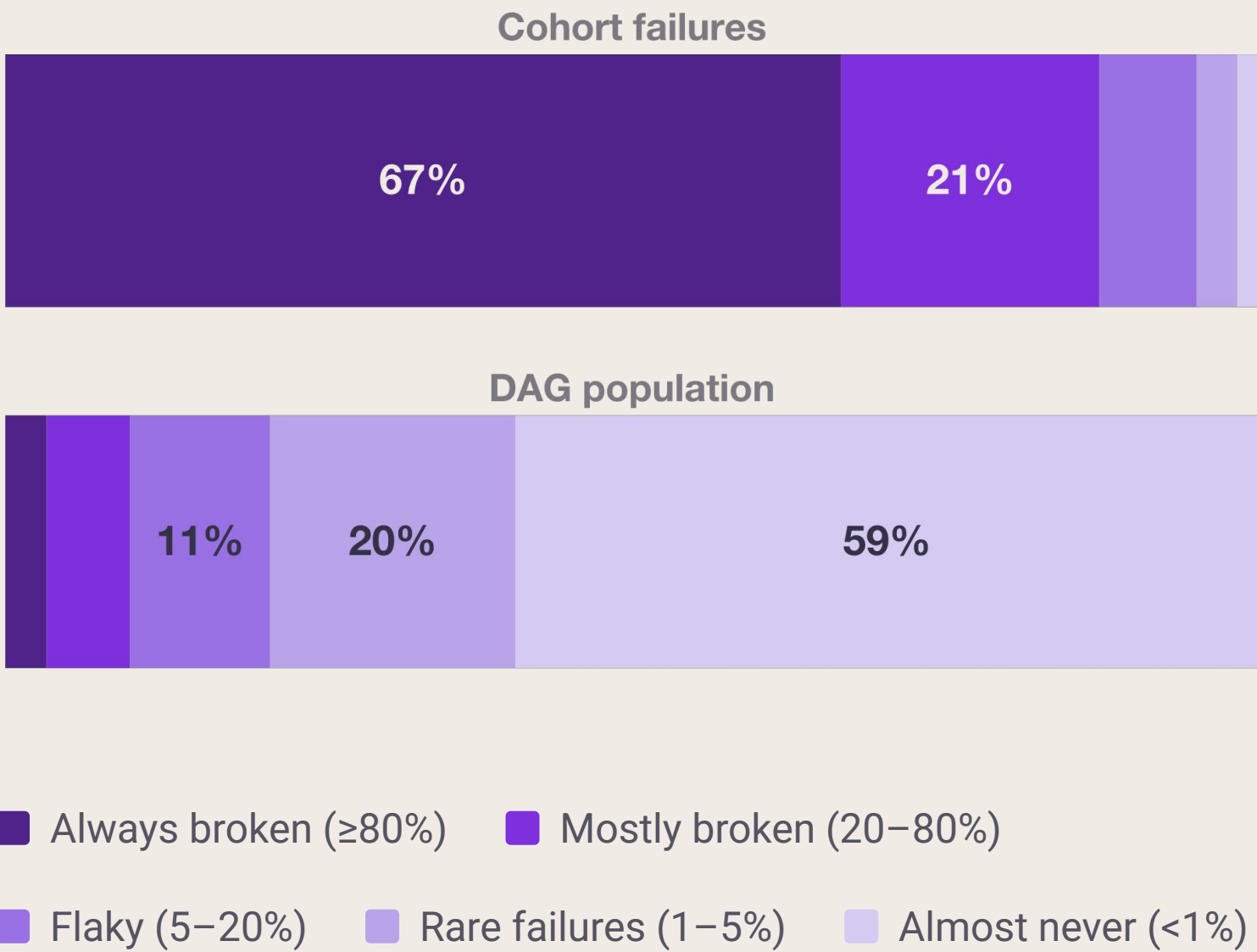
paying customers only

3% of DAGs generate 67% of failures.

59% almost never fail.

Cohort average: ~7% (automated runs only)

Strip the chronic failures and the rest runs at 2%.



WHERE THE FAILURES LIVE

~1/2

OF FAILURES

concentrated on one Astro Runtime
version

3-5×

FAILURE RATE

long-running vs. fast workloads

97%

FAILURE RATE

thousands of always-broken Dags

THE TEST

BEFORE

24 HOURS



DEPLOY

AFTER

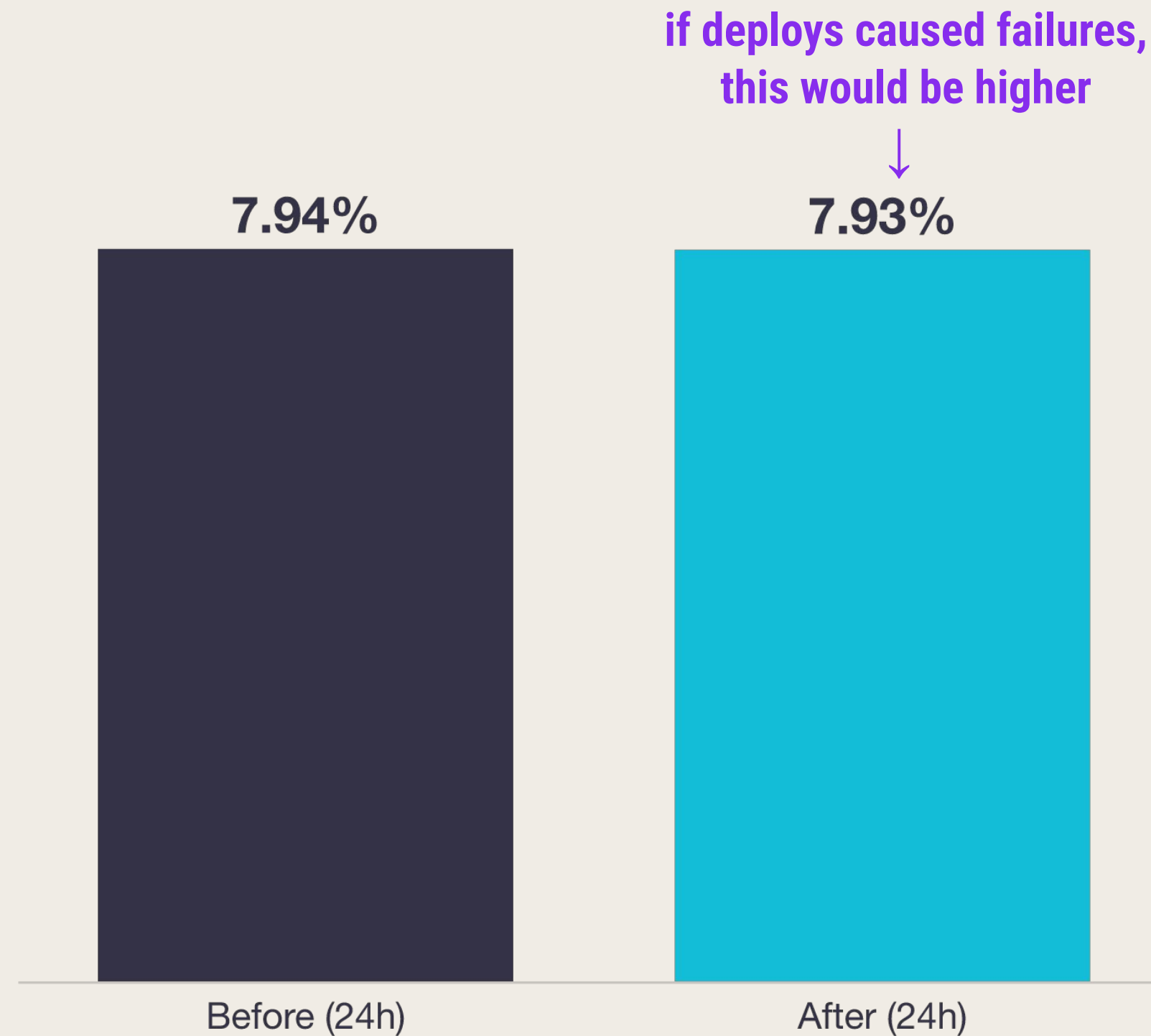
24 HOURS

same environment

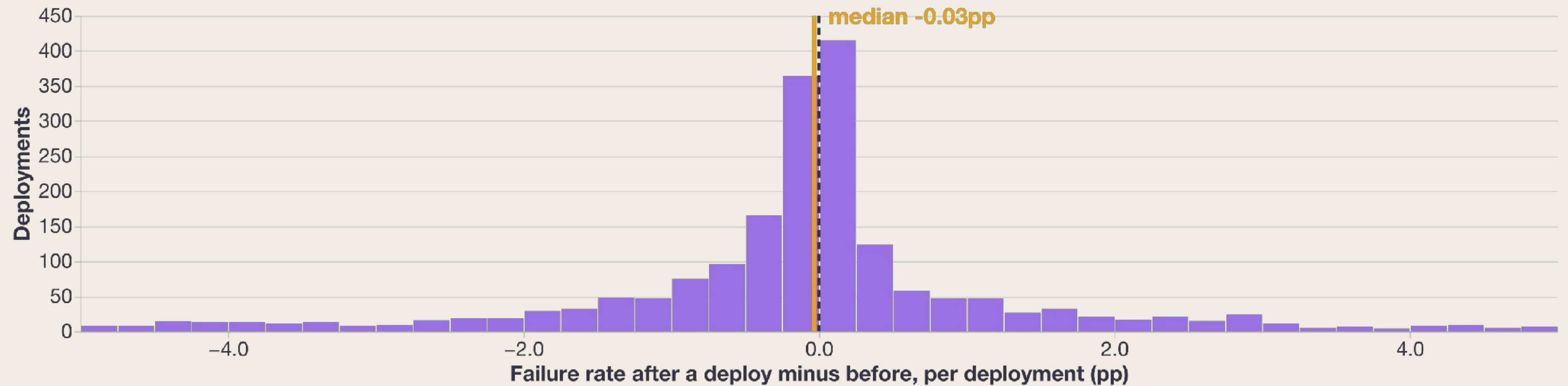
same Dags

adjacent windows

7.94% BEFORE A DEPLOY. 7.93% AFTER A DEPLOY.



Every environment, against its own before and after.



2,148 Deployments · median -0.03pp · 58% got better or stayed flat

THE SURVIVORSHIP OBJECTION

DEV
15.1%

PROD
6.6%

CI/CD PROTECTS PROD.

The deploy is an attention anchor.

- 08:15 dag_pipeline_sales · failed
- 09:47 dag_reporting_daily · passed
- 11:03 dag_pipeline_sales · failed

▲ 14:32 · Airflow 2.9 → 2.10

- 14:33 dag_pipeline_sales · failed

"it worked fine before 2.10!"

A CONVERSATION YOU'VE HAD

DAG 1: VENDOR API

`urllib3<2`

auth library pins it



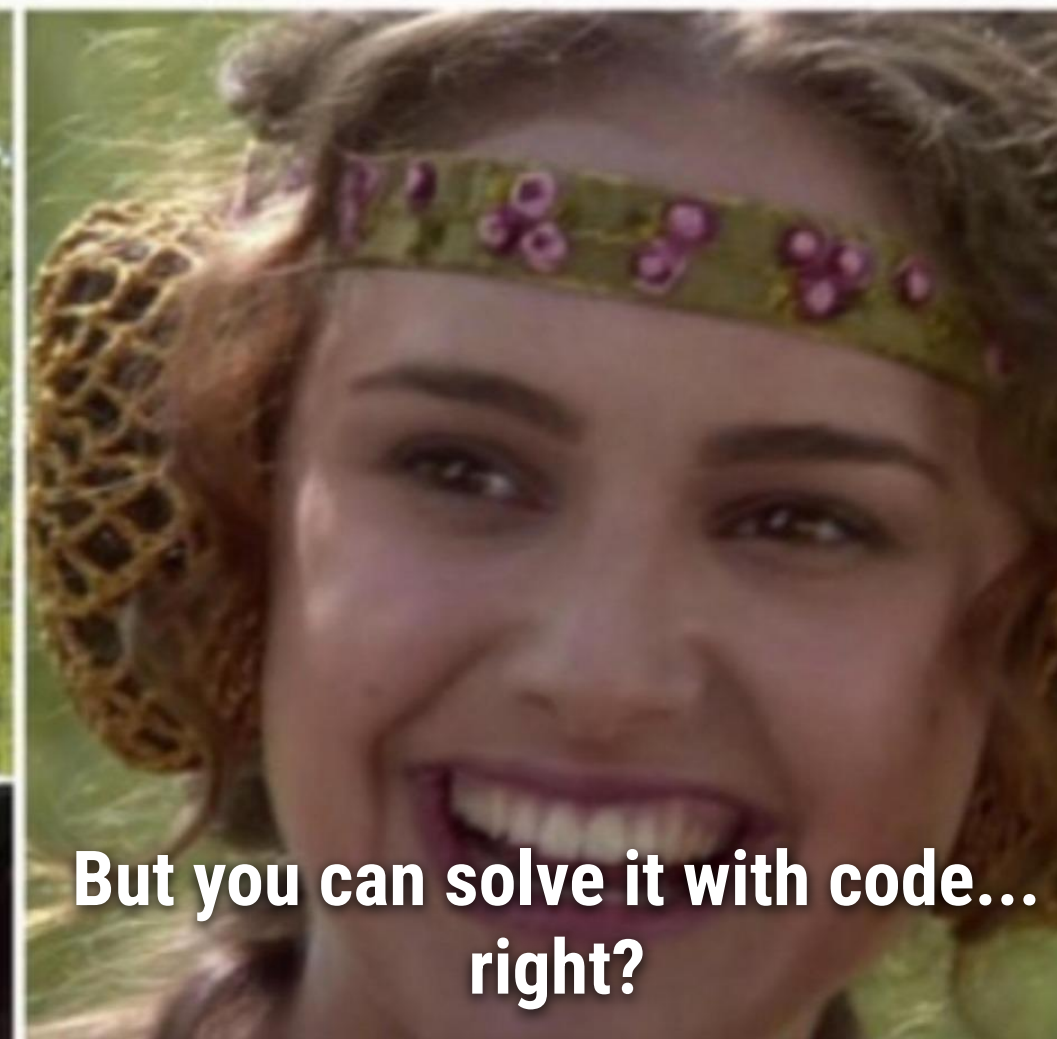
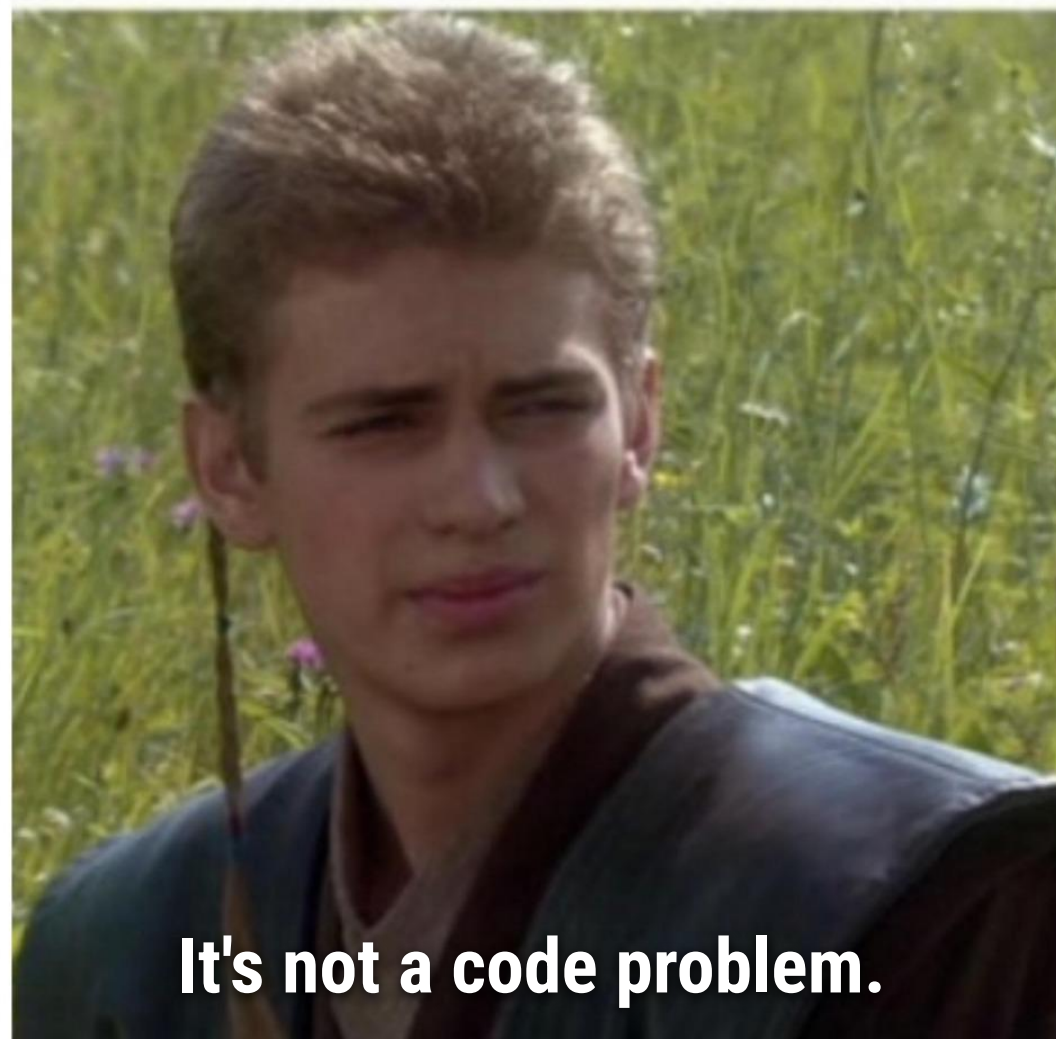
DAG 2: ML MODEL

`urllib3>=2`

model client requires it

"Just pin urllib3 and downgrade the model client, or refactor one of the DAGs to use a KPO."

senior platform engineer



Every check you built is green. **And then upstream drops a column.**

✓ code review 2 approvals

✓ pytest 247 tests passed

✓ DAG parse 0 import errors

✗ pipeline run `KeyError: 'user_type'`

The mental model that doesn't transfer

SOFTWARE ENGINEERING

CODE

↓ defines

INPUTS

↓ produce

OUTPUTS

Code drives outputs

vs

DATA ENGINEERING

INPUTS

↓ define

CODE

↓ produces

OUTPUTS

Data drives your code

Limitations

WHAT I MEASURED

the deploy boundary is flat

flat per environment too

dev fails more than prod

MY INFERENCE

schema drift

late arrivals

shifting distributions

02

WHO SHOULD IMPLEMENT AI?

THREE BOXES

	YOUR DATA PIPELINE	AN AGENTIC SYSTEM
UPSTREAM	Data you don't control	The prompt, the user input, the retrieved context
MIDDLE	Your code	The system prompt and the tools
DOWNSTREAM	An output somebody trusts	An output somebody trusts

Almost all of the tooling lives in **the middle one**.

The five layers you actually operate

	WHAT YOU RUN	WHAT AI TEAMS ARE BUILDING
1 CODE CORRECTNESS	Review, tests, lint, and the CI that runs them. What software engineering does well, and where your job starts rather than ends.	The same thing. This is the one layer that already transferred.
2 DATA CORRECTNESS	Schema checks, null and uniqueness assertions, dbt tests, Soda, SQL check operators, checks hand-rolled into a task. Written because the code was right and the data wasn't.	Eval suites
3 TIME CORRECTNESS	Freshness SLAs, arrival windows, completeness by cutoff. "Ran on schedule" and "processed fresh data" are different claims.	Drift and model-version monitoring
4 STATE SAFETY	Write-audit-publish, staging tables, publish gates. The run finished. Is the write safe for someone to read?	Guardrails and output validators, before a response reaches a user or a database
5 RECOVERY AND DEGRADATION	Quarantine paths, stale-but-labeled data, idempotent backfills. Trustable output when the input misbehaves.	Fallback responses, human review, quarantining a confidently wrong answer instead of serving it

One of these transferred. **Four are being rebuilt from scratch.**

Every layer sits on the same four rungs

	BLIND	DETECTING	ALERTING	GATING
1 · Code				
2 · Data				
3 · Time				
4 · State				
5 · Recovery				

Gating on layer 1, blind on most of the rest.

Your one reliable data quality check is **the CEO noticing**. Increasingly, nobody does.

Everything you can buy fires when **your code fails.**

WHAT AN ORCHESTRATOR CAN SEE

The task raised. The run overran. The process died. Retries, callbacks, trigger rules, deadline alerts all key off this, because execution is the only thing an orchestrator can observe.

WHAT IT CAN'T

The column changed meaning. The file was late. The numbers are plausible and wrong. Vendors will sell you detection here. Gating needs a threshold, and a threshold is a decision about what's acceptable at your company.

Software engineering never needed layers 2 through 5, so it never built them.

BUT YOU DID. USING AIRFLOW.



IN AIRFLOW TODAY

FOR AI TODAY

1 CODE

Review, tests, DAG parse checks in CI

The same. Already solved.

2 DATA

SQL check operators, dbt integration, assertions in a task

Structured-output constraints in the AI provider

3 TIME

Data intervals, assets, sensors, deadline alerts

Drift and model-version monitoring

4 STATE

Write-audit-publish as a DAG shape, publish gates as tasks

Human-in-the-loop operators

5 RECOVERY

Task retries, trigger rules, callbacks, backfill

Retry inside the harness, human review

ONE THING HERE IS NEW

DATA PIPELINE

AI PIPELINE

THE INPUTS

MOVES

MOVES

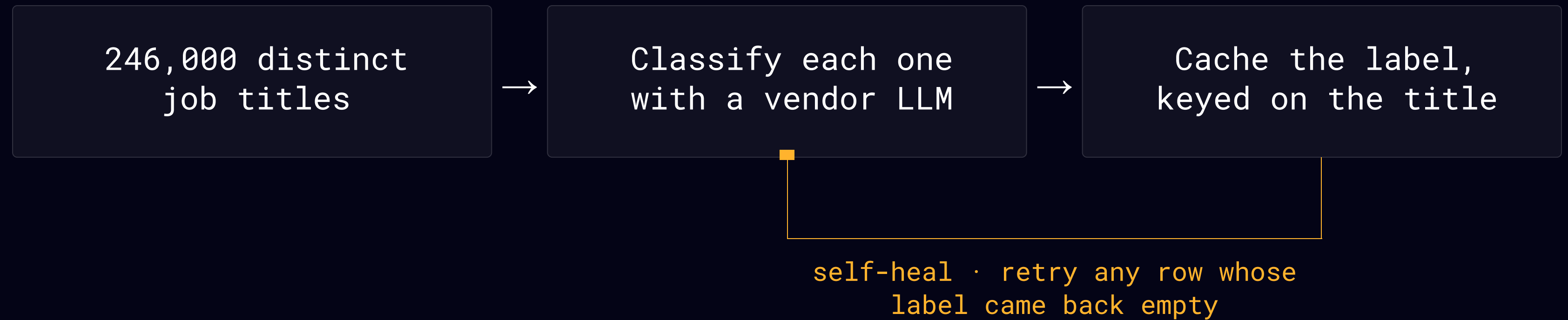
THE FUNCTION

FIXED

MOVES

One moving part. **Now two.**

THE DAG



THE CHANGE

CHANGE 1 · THE KEY

-	<code>classify(title): "labels"[0]</code>	"Data Engineer"
+	<code>classify(title): "label"</code>	NULL no error · task succeeds

CHANGE 2 · THE FALLBACK

-	<code>COALESCE(label, 'other')</code>	retry branch stays shut
+	<code>removed</code>	the only thing holding the retry branch back

THE RETRY BRANCH FIRED

246,000

ROWS PER PASS

96

PASSES PER DAY

4

DAYS UNDETECTED

Every pass, **a full set of billable model calls.**

WHAT THE PLATFORM REPORTED

	<	                                              	>>
check_self_heal_volume			
sf_account_log ▾	                                              		
sf_campaign_member_changes ▾			
sf_campaign_members ▾	                                              		
sf_contact_log ▾	                                              		
sf_contact_title_classification ▾	                                              		
dagset_done	                                              		

0

EXCEPTIONS RAISED

0

FAILED TASKS

100%

OF THE OUTPUT WRONG

THE ONE HARD RULE

**Never automate recovery before you can
tell good output from bad.**

WHAT THEY BUILT AFTERWARD

4 A MERGE GATE

Fail the run if more than 10% of the batch came back unclassified, so a poisoned batch never reaches the cache.

5 A CIRCUIT BREAKER

Hard-fail before the LLM calls go out if the retry branch would re-select more than 5% of the cache. Costs nothing to trip. Halts, and waits for a human.

2 STILL MISSING

Nothing can tell a good label from a bad one. Explicit trade-off as taxonomy isn't fixed.

THE AMENDMENT

Never automate recovery before you can tell good output from bad...

...unless you can bound the blast radius.

**After the fix, forty rows out of 246,000 were
permanently stuck.**

dsfh

qwe

...

JOB TITLES, AS TYPED

The model is forbidden from abstaining. Some answer on a later pass. A stubborn few never answer at all.

FROM THE COMMIT THAT CLOSED IT

“prompt engineering can't fix it”

The containment was to park anything that abstains twice, so it stops re-entering the retry pool.

40

ROWS — A ROUNDING ERROR

1,900

BILLABLE CALLS A DAY, INDEFINITELY

I've spent this talk saying you already know how to do this.

**Nobody knows how to evaluate
non-deterministic outputs at scale.**

Especially not me.

THE BET

If I run this same analysis on AI pipelines in two years, I'd bet on the same result.

03

THREE MOMENTS AND WHAT TO SAY

MOMENT 1 · THE POSTMORTEM

A pipeline failed, and someone asks **what we shipped**.

"It's almost never what we shipped. The question I'd ask first is **what changed upstream."**

Debugging failures you didn't cause is the job.

The AI project builds **its own reliability stack**.

"Evals are data quality tests. Model drift is schema drift. Guardrails are a publish gate. We run all three in production today — happy to bring them over so you don't have to build them twice."

You've been on call for all three. Same work, finally counted.

MOMENT 3 · THE RELIABILITY TARGET

A reliability number for a system **whose inputs you can't veto.**

"Give us a contract with the team upstream — or **measure us on the part we control."**

Every other team with a target owns its front door.

IF YOU RUN A DATA TEAM

Your team already runs the AI project's reliability layer. Nobody knows it.

Make it visible before someone builds it twice. The window is kickoff, not eighteen months in.

**You don't prevent failure. You
bound it.**

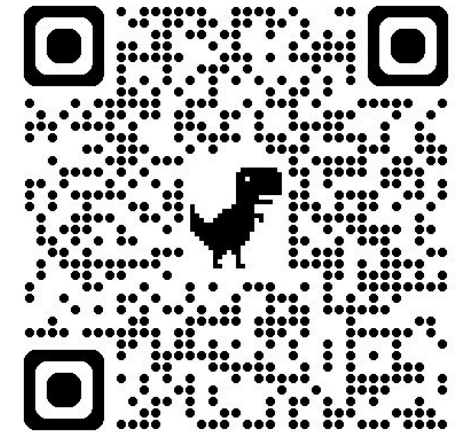
Agentic AI's reliability problem is an inputs problem. You've had it for a decade.

THANK YOU.
QUESTIONS?

LET'S KEEP IN TOUCH



Constance Martineau
LinkedIn



These slides
PDF · Google Drive